School of Computer Science & Engineering

B. Tech CSE GENERAL

Operating System

Topic: Activity Report: LOOK vs C-LOOK Disk Scheduling Simulator

Submitted to:

Dr. Puja Das
Assistant Professor,

Department of Computer Science and Engineering
Jain (Deemed-To-Be) University

Submitted by:

USN	Name
25BTRCN182	KUNJAL CHAUDHARY

Branch & Section:	**CSE - General**
Date of Submission:	**09/11/2026**
GitHub Repository:	https://github.com/KunZall/look-clook-simulator

Activity Report: LOOK vs C-LOOK Disk Scheduling Simulator

Source code: https://github.com/KunZall/look-clook-simulator

1. Aim

To set up and run an interactive web simulator of the **LOOK** and **C-LOOK** disk scheduling algorithms, and to compare their head movement, service order and performance on the same set of disk requests.

2. Objectives

- Understand how disk scheduling reduces seek time.
- Study how LOOK and C-LOOK service a request queue.
- Run the simulator, record the service sequence and total head movement for each algorithm, and verify the results by hand.
- Compare the two algorithms and state which suits which situation.

3. Background

Disk scheduling is the way an operating system orders pending read/write requests so the disk head moves as little as possible. The time the head takes to move to the right cylinder is the **seek time**, the slowest part of disk access, so a good order improves throughput.

LOOK: the head moves in one direction servicing requests, going only as far as the last request (not to the end of the disk). It then reverses and services the remaining requests on the way back.

C-LOOK (Circular LOOK): the head moves in one direction to the last request, then jumps to the lowest pending request at the other end and continues in the **same** direction. It never reverses, which gives more uniform waiting times.

	LOOK	**C-LOOK**
End of sweep	Reverses direction	Jumps to the other end
Service direction	Both ways	One way only
Waiting time	Less uniform	More uniform
Total movement	Often lower	Can be higher (includes the jump)

4. Tools and Technologies

Part	Technology	Purpose
Backend	Python 3.12, FastAPI, Pydantic	Algorithms, input validation, metrics, API endpoints
Testing	pytest	Unit tests for both algorithms
Frontend	React, TypeScript, Vite 5.4.21	Input form, animation, tables
Charts	Recharts	Statistics and comparison charts
Styling	CSS	Dark theme and layout
Environment	VS Code, Node.js v26.13.0, npm 11.20.0, Windows	Development and execution

5. System Design

The project is split into a backend and a frontend that talk over HTTP.

Flow: browser → `/api/compare` → Python runs both algorithms on identical input → steps and metrics are returned as JSON → the interface animates the head and fills the tables.

Backend file	Role
`algorithms/look.py`	LOOK: sweep in one direction, reverse at the last request
`algorithms/clook.py`	C-LOOK: sweep, then jump (marked as a jump step) and continue
`algorithms/common.py`	Shared `build_result()` that computes steps, total, jump total, average seek, maximum seek
`services/simulation_service.py`	Input validation and `compare()`
`main.py`	FastAPI endpoints `/api/simulate` and `/api/compare`
`tests/test_algorithms.py`	pytest checks for correctness

The only real difference between the two algorithms is the order of the second group of requests (those behind the head): LOOK sorts them descending, C-LOOK sorts them ascending and marks the first move as a jump. Sorting dominates the cost, so both run in O(n log n).

Input validation rejects: minimum cylinder not below maximum, an empty queue, more than 100 requests, a head or request outside the cylinder range, and an invalid direction.

6. Procedure

1. Unzipped the project and opened the folder in VS Code.
2. Backend: `cd backend`, `pip install -r requirements.txt`, then `python -m uvicorn main:app --reload --port 8000`. The server reported it was running on `http://127.0.0.1:8000`.
3. Confirmed the API documentation page at `http://127.0.0.1:8000/docs` lists `/api/simulate` and `/api/compare`.
4. Frontend: `cd frontend`, `npm install`, `npm run dev`. Vite served the app at `http://localhost:5173`.
5. Entered the inputs (head 53, range 0 to 199, queue 98, 183, 37, 122, 14, 124, 65, 67, direction right) and ran the simulation.
6. Recorded the service sequence and metrics, then checked them by hand.

Problems faced and how they were solved

Problem	Cause	Solution
`uvicorn` not recognized	pip's Scripts folder is not on PATH	Ran it as `python -m uvicorn main:app --reload --port 8000`
`npm` not recognized	Node.js was not installed	Installed Node.js LTS and restarted VS Code
`npm.ps1` cannot be loaded	PowerShell script execution was disabled	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
Vite error: failed to resolve `./api`	`src/api.ts` was not found by Vite	Recreated `api.ts` with the request types and the `compare()` function
"Unexpected end of JSON input"	No proxy to the backend (`vite.config.ts` missing; `localhost` can resolve to IPv6 while the backend listens on IPv4)	Added `vite.config.ts` proxying `/api` and `/ws` to `127.0.0.1:8000`, then restarted the frontend
C# .NET SDK popup in VS Code	The C# extension looks for .NET	Ignored; the project does not use C#

7. Experiments and Results

Setup (both experiments): head = 53, cylinders 0 to 199, queue = 98, 183, 37, 122, 14, 124, 65, 67.

Experiment 1: direction right (run in the simulator)

Step	LOOK move	Distance	C-LOOK move	Distance
1	53 → 65	12	53 → 65	12
2	65 → 67	2	65 → 67	2
3	67 → 98	31	67 → 98	31
4	98 → 122	24	98 → 122	24
5	122 → 124	2	122 → 124	2
6	124 → 183	59	124 → 183	59
7	183 → 37	146	183 → 14 (jump)	169
8	37 → 14	23	14 → 37	23
Total		**299**		**322**

Metric	LOOK	C-LOOK
Service sequence	65, 67, 98, 122, 124, 183, 37, 14	65, 67, 98, 122, 124, 183, 14, 37
Total head movement	299	322
Jump distance	0	169
Average seek (total ÷ 8)	37.38	40.25
Maximum single move	146	169
Final head position	14	37

C-LOOK moved 23 cylinders more than LOOK, which is 7.69% more.

Hand check, LOOK: (183 − 53) + (183 − 14) = 130 + 169 = 299. C-LOOK: 130 + (183 − 14) + (37 − 14) = 130 + 169 + 23 = 322.

Experiment 2: direction left (same inputs, computed with the backend functions)

Metric	LOOK	C-LOOK
Service sequence	37, 14, 65, 67, 98, 122, 124, 183	37, 14, 183, 124, 122, 98, 67, 65
Total head movement	208	326
Jump distance	0	169
Average seek (total ÷ 8)	26.00	40.75
Maximum single move	59	169

Here C-LOOK moved 118 cylinders more than LOOK, which is 56.73% more. These values match the project's unit tests, which assert LOOK = 208 and C-LOOK = 39 + 169 + 118 = 326.

8. Graphs

The graphs below are drawn from the results above.

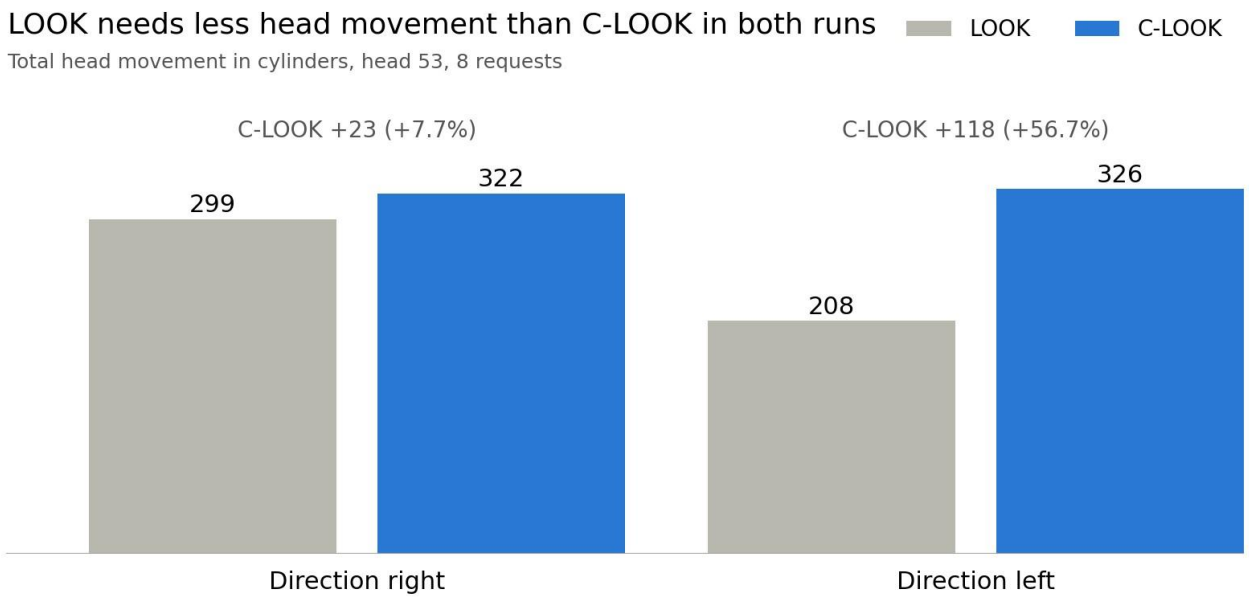


Figure 1. Total head movement of LOOK and C-LOOK in the two experiments.

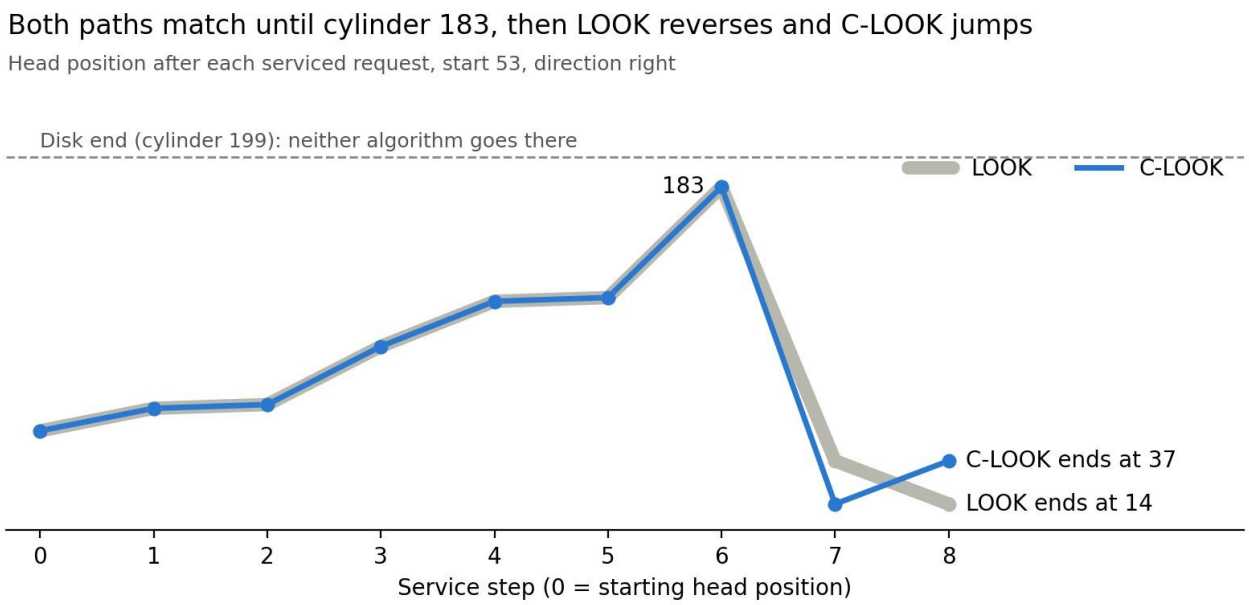


Figure 2. Head position after each serviced request in Experiment 1. The paths overlap until cylinder 183, then split.

9. Observations

1. For direction right, both algorithms serviced requests in the same order up to cylinder 183 (65, 67, 98, 122, 124, 183). They differ only afterwards.
2. LOOK reversed at 183 and serviced 37 then 14. C-LOOK jumped from 183 to 14 (the red dashed line in the simulator) and then moved up to 37.
3. Neither algorithm travelled to the physical end of the disk (cylinder 199). Both turned at the last request, 183, which is what distinguishes the LOOK family from SCAN and C-SCAN.
4. LOOK had the lower total movement in both experiments (299 vs 322 going right, 208 vs 326 going left).
5. C-LOOK's long jump (169 cylinders) dominates its cost and sets its largest single move, whereas LOOK's largest move was 146 (right) and only 59 (left).
6. The direction of the first sweep changed the results noticeably. The gap between the algorithms was small going right (7.69%) and large going left (56.73%), because only one request (14 and 37) lay behind the head on the left side and a large group lay ahead of it.
7. The head ended at different cylinders: 14 for LOOK and 37 for C-LOOK (direction right).
8. Every request was serviced exactly once by both algorithms, and the simulator results matched the hand calculations and the pytest tests.

10. Conclusions

- LOOK and C-LOOK both avoid the wasted travel of SCAN and C-SCAN by turning at the last request.

- In these experiments LOOK gave less total head movement, so it is the more efficient choice for this request pattern.
- C-LOOK pays extra movement for the return jump, but it services requests in one direction only, which gives more uniform waiting times. It is preferable when requests arrive continuously and fairness matters more than minimum total movement.
- The better algorithm depends on where the requests lie relative to the head and on the starting direction, so no single algorithm wins in every case.
- Separating the algorithms, validation and web layers made the code easy to test, and the simulator reproduced the expected textbook values of 299 and 322.

11. Limitations and Future Work

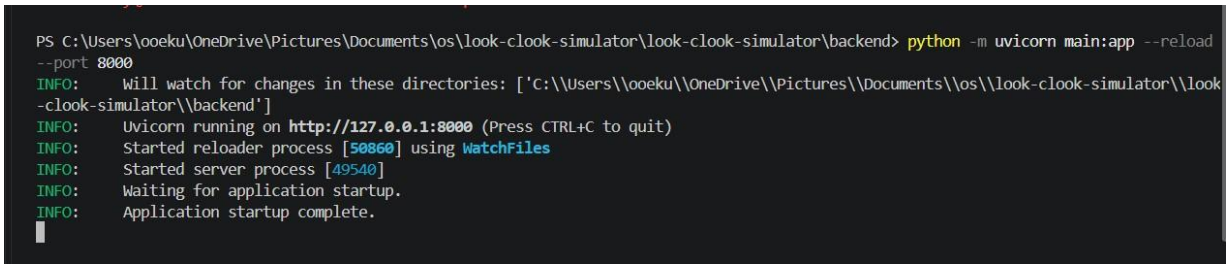
- The simulator works on a fixed queue and does not model requests that arrive while the head is moving.
- It reports head movement only, not rotational latency or real transfer time.
- Future work: add FCFS, SSTF, SCAN and C-SCAN for a wider comparison, and support random request generation and exporting results.

12. References

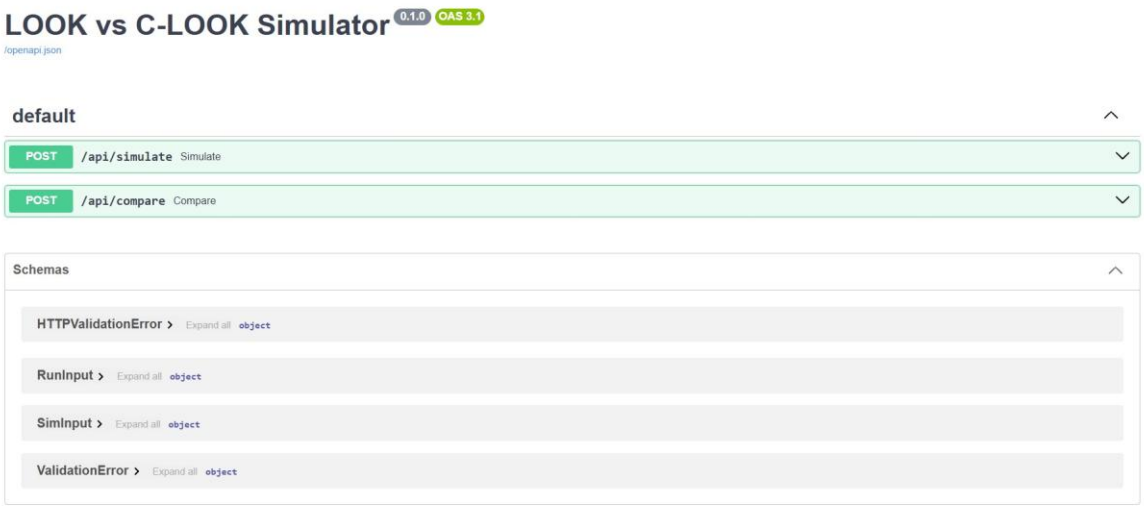
1. Silberschatz, Galvin and Gagne, *Operating System Concepts*, chapter on mass-storage structure (disk scheduling).
2. FastAPI documentation, https://fastapi.tiangolo.com
3. React documentation, https://react.dev and Vite documentation, https://vitejs.dev
4. Project source code (GitHub repository), https://github.com/KunZall/look-clook-simulator

13. Screenshots

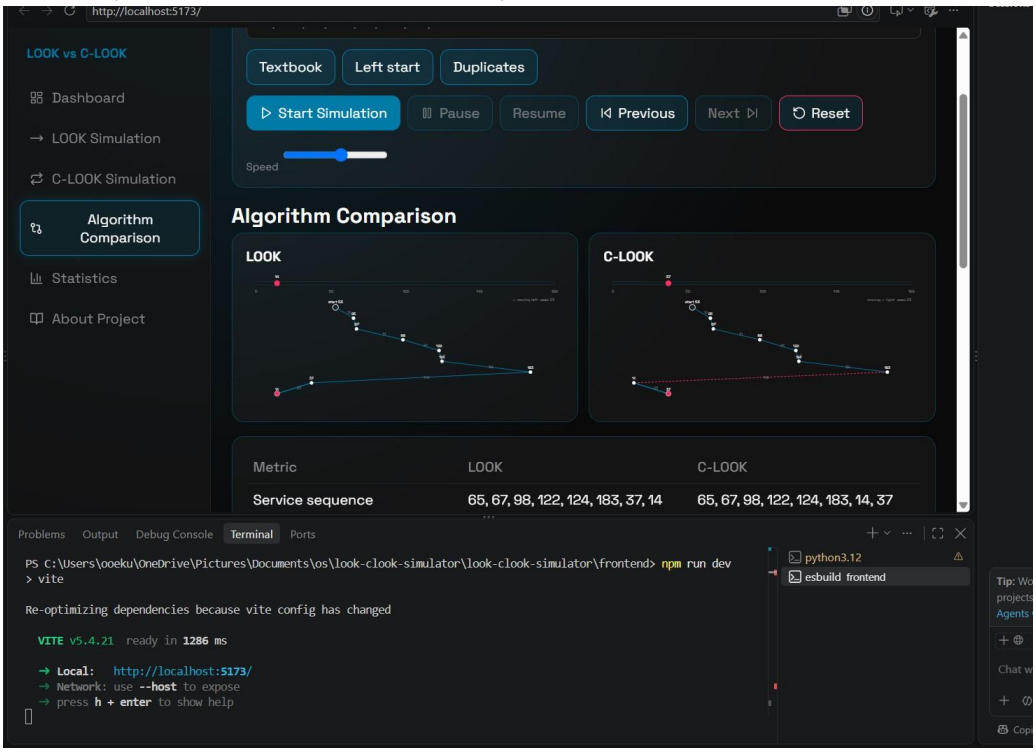
The screenshots from the run are shown below.



Screenshot 1. The FastAPI backend started with `python -m uvicorn main:app --reload --port 8000`*. Uvicorn reports it is running on http://127.0.0.1:8000 and application startup is complete.*



Screenshot 2. The automatic API documentation page at `/docs`*, listing the* `/api/simulate` *and* `/api/compare` *endpoints and their schemas. This confirms the backend works.*



Screenshot 3. The Algorithm Comparison page of the simulator for the textbook input. The paths for LOOK (left) and C-LOOK (right) are identical until cylinder 183. LOOK then reverses to 37 and 14, while C-LOOK jumps (red dashed line) to 14 and moves up to 37. The service sequence row confirms the order 65, 67, 98, 122, 124, 183, 37, 14 for LOOK and 65, 67, 98, 122, 124, 183, 14, 37 for C-LOOK.